

Rubik's Cube USRA Project

Benjamin Kyle
Concordia University

Supervisors:

Prof. Tiberiu Popa, Prof. Denis Pankratov

<https://github.com/cosmic-bkyle/usra-project>

1 Introduction

The Rubik's Cube is a puzzle that many love to solve as a hobby. The World Cube Association governs official competitions, in which there are 18 different puzzle events. One of the events is 3x3x3 Fewest Moves, where competitors are given one hour to search for a solution using the fewest moves. For reference, world-class competitors for the standard 3x3x3 Cube event use solutions of around 55 moves to solve the puzzle in 4 seconds. This means that they're consistently executing over 12 turns per second. Less known is the 3x3x3 Fewest Moves event, referred to by its community as the Fewest Moves Challenge (FMC). In its exam-style format, competitors are given a scramble sentence in the language of the cube, unlimited pens and paper sheets, and three standard Rubik's Cubes. In 1 hour, they produce a written solution sentence which, when applied after the scramble sentence, yields a solved cube. World-class competitors are averaging between 21-22 moves.

It is important to note that there are 43,252,003,274,489,856,000 unique legal permutations of the cube's stickers, but, somewhat surprisingly, every position can be solved in 20 moves or fewer^[1]. That said, finding the optimal solution is difficult, in fact NP-complete^[2]. The NP-hardness of puzzles like Sudoku, Minesweeper, and even Mario Bros is why we humans take pleasure in solving them. We understand that the best we can do is develop rules of thumb, known as heuristics. Since heuristics are by definition non-optimal, there are always speculations in puzzle communities about what are the best ones, leaving some room for "style choices" and adding to the fun. Contributing to the pool of heuristics for the Fewest Moves Challenge was a high-level goal of this summer research project.

Several machine learning methods have before been implemented on the Rubik's Cube. Researchers' goals have largely been to train black-box models to solve the cube in the fewest moves. Impressively, The CayleyPy Project^[3] used a combination of deep learning and beam search to solve the cube optimally 98% of the time, which is evidence that machines can learn efficient non-interpretable strategies. As a competitor of the Fewest Moves Challenge, I am personally interested in evaluating the correlations between human-recognizable features of cube positions and distances to the solved state. This naturally suggests the most ubiquitous machine learning method, linear regression. If I can leverage intuition to decide the features beforehand, then throwing them into linear models and looking at the weights can immediately influence my competition strategy.

Solving the Rubik's Cube is a graph search problem: to traverse from the "scramble" node to the "solved" node on the Cayley graph of the Rubik's Cube group. The method that is used by the world's best competitors is a two-phase search algorithm called Domino Reduction (DR). The first phase reduces the puzzle into the domino subgroup (see Section 2), and the second phase brings the new puzzle to the solved state. In this article I refer to the first phase as scrambled→domino and the second phase as domino→solved. The tasks are very distinct from an algorithmic standpoint: the first ends in a choice of millions of "checkpoint" states, and the second must end in the solved state. As competitors can find multiple distinct solutions to scrambled→domino in the first half of the hour, being able to *predict the length of domino→solved* gives a ranking of all the candidate checkpoints, telling the competitor how to allocate their remaining time. Linear methods for this prediction task are outlined in section 3. Non-linear methods via modified feedforward networks are in Section 4, with weaker results. Section 5 discusses next steps, which are to apply the same research methods on scrambled→domino. Section 6 concludes and mentions some personal anecdotes.

2 Preliminaries / Cubing Background

- **FMC**: Fewest Moves Challenge
- Rubik’s Cube legal moves: $\{F, F', F2, R, R', R2, L, L', L2, B, B', B2, D, D', D2, U, U', U2\}$
- **EO**: Edge Orientation. The first step for most competitors’ FMC strategies. WLOG all possible states with edges oriented are generated by $\{U, D, R, L\}$.
- **DR**: Domino Reduction. The next major reduction of the cube after EO. There are several procedural strategies to go from EO to DR. All domino states are generated by $\{U, D, R2, L2, F2, B2\}$
- **HTR**: Half-Turn Reduction. Once performed, the cube is solvable with $\{F2, B2, R2, L2, D2, U2\}$, which conversely generates all HTR states. Typically, from HTR humans solve all but the E-slice (“leave slice”). This is because one may backtrack through the domino solution and insert slice moves to solve the E slice with small cost (~ 1 move).

3 Domino→Solved, Linear

3.1 Naive Adjacency Model

Early in the research period we discussed G , a graph which represents the adjacencies of physical pieces on a cube’s state:

$$G = (U, V, E, w)$$

where

U = corner pieces of the cube

V = border pieces of the cube

$E = \{(u, v) \mid u \text{ is touching } v \text{ in the cube state}\}$

$w : E \rightarrow \mathbb{R}$ assigns weights to the edges.

Feeding such graphs G computed from Nissy-generated domino scrambles to a basic Lasso model from scikit learn, we find the following weights, with the bias at 13.XX.

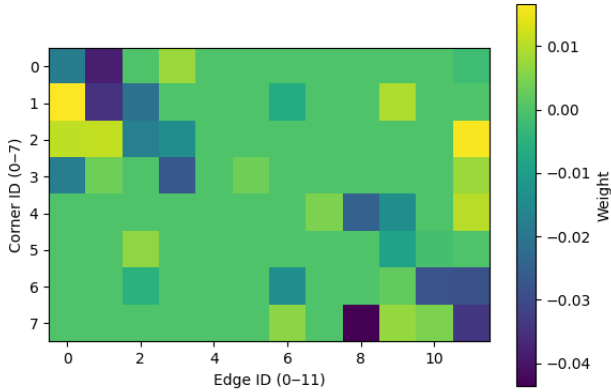


Figure 1: Learned weights of corner-edge relationships in 100k domino scrambles. Details of the scikit Lasso model used can be found in the source code.

3.2 Feature Engineering

3.3 Input

3.4 Hyperparameters

3.5 Results

4 Domino→Solved, Nonlinear

5 Next Steps: Scrambled→Domino

6 Conclusion and Personal Anecdotes

Notes

We aim to learn features of the Rubik's Cube states that correlate with promising optimal solution lengths. We are specifically interested in efficient trajectories from the Rubik's Cube group into structured subgroups. We will apply ML techniques to solutions generated by Nissy and GAP-based (Groups, Algorithms, Programming library) tools to improve human heuristics for the Fewest Moves Challenge. The following are some other research directions of our initial brainstorming:

- using ML to predict how many states are visited (time cost) in a human-heuristic-algorithm's search for $\leq$ optimal +1 for solving from the DR state to solved state
- helping humans with scrambled $\rightarrow$ DR without passing through EO so early
- using ML to predict an optimal first move on a scrambled cube / fmc substep
- speeding up solvers e.g. Nissy, Mallard
- (group-theoretic) discover properties of general algorithms which take random group elements to the nearest element of a particular subgroup, until the identity element is reached. (In the case of the cube and the current FMC meta, this is: Scrambled $\rightarrow$ EO $\rightarrow$ DR $\rightarrow$ HTR $\rightarrow$ Solved)

Related papers:

https://deepcube.igb.uci.edu/static/files/SolvingTheRubiksCubeWithDeepReinforcementLearningAndSearch_Final.pdf uses quarter-turn metric and deep learning to find efficient solutions.

<https://arxiv.org/pdf/2502.13266> A Machine Learning Approach That Beats Large Rubik's Cubes

<https://github.com/enricotenuti/h48thesis/blob/main/thesis.pdf> Computational analysis of Nissy

https://jerrybryan.com/rubik_2025.pdf Counting cube positions

Research Plan

Short-term Goals

My immediate short-term goals are outlined below.

1. Generate HTR subset stats, where the subsets are defined by chosen features: optimal corner solution length, htr subset.
2. Get my work environment set up: have a visual representation of the cube states, affectable by move sequence strings. In addition, support nissy commands in the same environment
3. Enact machine learning models (e.g. Supervised Regression), and FMC steps along with cube features suspected to correlate with good optimals for the steps (e.g. blockiness for DR $\rightarrow$ solved)
4. Discuss how to get the machine to learn the features (Deep learning)

Long-term Goals

The project's long-term goals are outlined below.

1. Write a report of findings
2. In the discovery of such important features, contribute to the strategies of world-class FMC competitors

Research Journal

May 20

- Discussed high-level goals of the project
- Reviewed "HTR Subset Stats" and asked:

- What distribution on DR states is the data from?
- Can I replicate the same numbers?
- What other features can we think of to predict the optimal solution length from the DR?
- Task for next week: Compute similar statistics for two other features; corner solution length and blockiness

May 27

- Reviewed code and the statistics generated for HTR subset X optimal length, as well as corner optimal length X optimal length
- Discussed G , a new approach towards the blockiness feature:

$$G = (U, V, E, w)$$

where

U = corner pieces of the cube

V = border pieces of the cube

$E = \{(u, v) \mid u \text{ is touching } v \text{ in the cube state}\}$

$w : E \rightarrow \mathbb{R}$ assigns weights to the edges.

- The graph is undirected. Observe that without the weights, every DR state corresponds to a unique such graph
- We would like the total weight of the graph to represent the quality of the DR state, in terms of its optimal solution length.
- If we could learn an algorithm to weigh the edges, then we could see what kind of blocks (e.g. RGY connected to GY) correlate with short solutions.
- Task for the week: develop a representation of this graph, given a dr scramble string. Try to run a linear regression to fill weights.

June 3

- Met with Denis and reviewed code from the week:
 - An object-oriented cube representation, affectable by input move sequences.
 - The bipartite adjacency graph of any given cube state
 - The generation of the dataset ($X = 100k$ bipartite graphs, $y =$ solution lengths)
- Some problems encountered include:
 - Segmentation faults on nissy's end while generating the data.
 - Some relationships were less prominent even though they are equivalent up to symmetries to ones which were strong.

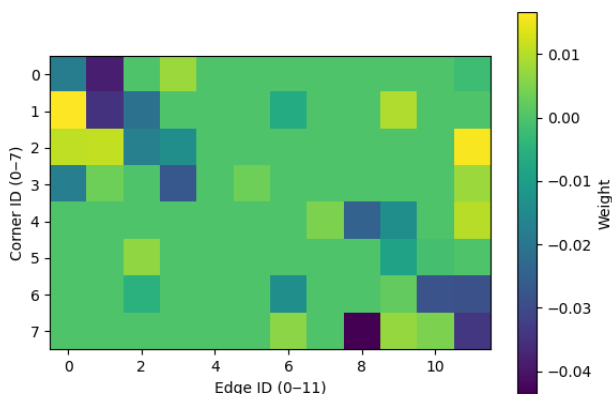


Figure 2: Learned weights of corner-edge relationships in 100k domino scrambles. Details of the scikit Lasso model used can be found in the source code.

- This week: Set up a more classic ML pipeline. Generate 300k scrambles and partition into train, val, and test sets. Tune hyperparameters / model selection by residual error.
- Implement modified cube state representation and adjacency graphs ignoring middle slice.
- The next feature will be triple-blocks, of which there exist two shapes (ignore E-slice). Try and throw these into the regression and compare their weights.

June 10

- Met with Tiberiu to discuss:
 - Appropriate test/train split (valid not necessary)
 - The program to count blockiness features
- Using blocky features only, the mean actual error is roughly 0.85. The weights learned are roughly:
 - Pairs: -0.16
 - Good L: -0.05
 - Bad L: 0.012
- This week, I grow dataset to 1 million scrambles. I try to learn a scoring function from the training set which minimizes residual error. This scoring function would be human-computible and so if it's within 0.5 moves of ground truth I consider that a success.
- In the weeks following, we venture into deep learning as in the CayleyProject paper, adapting to FMC substeps.

June 17

- Learned domino optimal guessing function, depicted below.
- we will now venture into neural networks, as done in the CayleyProject paper. Task for the week: get started using pytorch, play with small examples. Decide on gameplan.

June 26

- Idea 1: predict which of the 6 axes is closest to domino for human methods.
- Idea 2: Mimic the CayleyPy project with an MLP and a beam search, this time to find domino (fix an axis).

June 27

- Went over the first pytorch neural net implementation to predict the domino $\rightarrow$ solved distance.
- The input feature vectors were 208-dimensional binary vectors describing the cube's permutation. $206 = 8 \text{ corners} \times 8 \text{ corners} + 12 \text{ edges} \times 12 \text{ edges}$
- the observed mean actual error was around .84, better than the blocks feature but not better than the entire linear regression scoring function from previous weeks, on my engineered features.
- One training run observed mean actual error rising between epochs, so I should print the loss as well to ensure that this isn't a bug.
- A first task for this week is to try and improve the guess by just give the features that I engineered, no permutation, and experimenting with more epochs. Also messing around with the model architecture to better suit cubes.
- Next is to brainstorm convolutional networks - decide what kind of piece relations are analogous to e.g. 2x2 pixel regions of a photograph, to define potential convolutional layers.

July 4

- Discussed ideas for convolutional networks: project onto a 2d grid (and lose info), or remove meaningless connections among the layers.
- Denis proposes feedforward layers with nodes representing blocks of increasing size. Important question: would the learned weights be all the same due to symmetries?

Aug 1

- Discussed last competition's performance: 24, 26, DNF. Missed a 20 move solution from an 8 domino that I had written down. I weighed the features of a different domino too highly. A new type of feature was present on it, which I call "UFB", and which is an intuitively good indicator, but should be thrown into the linear regression to be weighed more accurately.
- Inspected code for the latest "convolutional" neural net, which uses a mask to customize connections. The input layer represents the stickers, and the hidden layers represent pairs, triples, and quadruples, sequentially. Results are poor, at .88 MAE. Three fully connected layers have been added just before the output but results do not improve.
- Troubleshooting for the neural net:
 - Feed the permutation and 8-10 fully connected layers to see if a deep feedforward can perform well
 - Rotate the data; select a random orientation for each scramble
- New direction: Feature engineering for scrambled $\rightarrow$ domino. Possible prediction tasks:
 - Given a scramble, which of the 6 axes is most fruitful (i.e. closest to the shortest dominoes)?
 - What is the value of one premove on a "doomed" scramble?
 - A DR requires EO on two axes. Which combinations of two adjacent EO cases are likely to give the most RZPs?
 - How much does HTR subset matter at the beginning of a solve? Which EO solutions are most likely to affect it? How much does RZP $\rightarrow$ DR affect it? Which cases
 - What is the value of one premove on a "doomed" scramble?